

Estatística Aplicada à Pesca e Aquicultura

Guia de estudo com R, RStudio e Quarto — apoiado por IA

Prof.~Evaldo Silva

Sumário

Boas-vindas	7
Como este curso funciona	7
Você não precisa já saber programar	7
O que você vai conseguir fazer ao final	7
Como navegar	8

I Fundamentos: R, RStudio e o fluxo de trabalho

1	Por que R nesta disciplina	11
1.1	O papel da IA aqui	11
1.2	O que vem a seguir	11
2	RStudio e Projetos	13
2.1	O painel do RStudio em 1 minuto	13
2.2	Projetos: o fim dos arquivos perdidos	13
2.3	Peça à IA, mas confira	13
3	Objetos, dados e pacotes	15
3.1	A seta de atribuição	15
3.2	Vetor: vários valores do mesmo tipo	15
3.3	Data frame: a sua tabela de coleta	15
3.4	Fator: variáveis categóricas	15
3.5	Pacotes: funções extras	16
4	Quarto e saída Typst	17
4.1	O que é Quarto	17
4.2	Anatomia de um arquivo <code>.qmd</code>	17
4.3	Renderizar	17
4.4	Figuras e tabelas com legenda e referência	18
5	IA no RStudio (Posit Assistant)	19
5.1	Como pedir bem	19
5.2	O ciclo: pedir → ler → validar → ajustar	19
5.3	Erros honestos que a IA comete	19
5.4	O que continua sendo só seu	19

II Pilar 1 — Planejamento amostral e experimental

6	Pilar 1 — Por que planejar antes de coletar	23
6.1	As perguntas do planejamento	23
6.2	O que vem nos próximos capítulos	23
7	Amostragem em campo e entrevistas	25
7.1	Tipos de amostragem	25
7.2	Sortear no R	25
7.3	Entrevistas com pescadores e comunidades	25
7.4	Tamanho amostral: a ideia	25
8	Planejamento de experimentos	27
8.1	Pseudo-replicação: o erro clássico	27
8.2	Delineamentos comuns	27
8.3	Casualizar no R	27
8.4	Do delineamento à análise	27

III

Pilar 2 — Análise com R

9	Pilar 2 — Importar dados	31
9.1	Antes de importar: organize a planilha	31
9.2	Importar CSV	31
9.3	Importar Excel (.xlsx)	31
9.4	A primeira coisa a fazer depois de importar	31
10	Tratamento de dados	33
10.1	O operador pipe	33
10.2	Os verbos do dplyr	33
10.3	Resumir por grupo	33
10.4	Lidar com valores ausentes e categorias inconsistentes	33
10.5	O resultado tratado alimenta o Pilar 3	34
11	Pressupostos das análises	35
11.1	Os pressupostos mais comuns	35
11.2	Verificar normalidade	35
11.3	Verificar homogeneidade de variâncias	35
11.4	E quando os pressupostos falham?	35
11.5	O fluxo correto	36
12	As análises	37
12.1	Mapa de decisão	37
12.2	Comparar dois grupos	37
12.3	Comparar três ou mais grupos: ANOVA	37
12.4	Relação entre variáveis numéricas: regressão	37

12.5	Variáveis categóricas: qui-quadrado	38
12.6	Reportar o resultado	38

IV **Pilar 3 — Comunicação de resultados**

13	Pilar 3 — Comunicar resultados	41
13.1	O princípio do documento único	41
13.2	Estrutura de um relatório de aula prática	41
13.3	Tabelas e figuras que se referenciam sozinhas	42
13.4	Citar a literatura	42
13.5	Renderizar para entrega	42

V **Desafios com dados**

14	Desafios com dados	45
14.1	Como entregar um desafio	45
14.2	Desafio 1 — Peso entre dois estuários (nível inicial)	45
14.3	Desafio 2 — Dieta e crescimento (nível intermediário)	45
14.4	Desafio 3 — Relação peso-comprimento (nível intermediário)	45
14.5	Desafio 4 — Percepção dos pescadores (dados de entrevista)	45
14.6	Autoavaliação	45
	Referências	47
	Livros-guia da ementa	47
	Bibliografia	49

Boas-vindas

Este é o **guia de estudo** da disciplina **Estatística Aplicada à Pesca e Aquicultura** (EP05042, 45 h) do Bacharelado em Engenharia de Pesca da UFPA — Campus Bragança (FEPESCA / IECOS).

Aqui você encontra a teoria, os roteiros de código em R e os exemplos de publicação que conversam com as **apresentações de aula**, com os **vídeos do canal no YouTube** e com os **desafios de dados**. É o seu ponto de retorno: sempre que algo não ficar claro na aula ou no vídeo, volte para o capítulo correspondente.

Como este curso funciona

A disciplina se apoia em **três pilares**, e tudo o que fazemos cai em um deles:

1. **Planejamento** — como decidir *o que e quanto* coletar antes de ir a campo: amostragem de organismos (água, manguezal, embarcações), entrevistas com pescadores e comunidades, e experimentos com pescado ou ecológicos.
2. **Análise com R** — importar, tratar, **verificar pressupostos** e rodar as análises, deixando os dados prontos para comunicar.
3. **Comunicação** — transformar resultados em algo publicável: o que vai no artigo, no TC, na dissertação, na tese ou no relatório de aula prática, via **Quarto com saída Typst**.

Parte do conteúdo você estudará de forma **autodidata**, pelos vídeos. Parte veremos juntos em aula: os **conceitos**, por que a estatística importa e os **pressupostos** de cada análise. E uma parte final é praticar a **publicação**.

A partir de 2026 temos IA integrada ao RStudio (o **Posit Assistant**). Neste curso usamos IA de forma consciente, sob um pacto simples:

A IA escreve o código. Você valida os pressupostos e interpreta o resultado.

Uma IA roda um teste-t num piscar de olhos — inclusive quando os pressupostos foram violados e o teste não deveria ser usado. *Reconhecer isso é competência humana, e é o que avaliamos.* Você não precisa decorar sintaxe; precisa saber o que pedir, ler a resposta com olhar crítico e julgar se o resultado faz sentido biológico e estatístico.

Você não precisa já saber programar

Você vem de **Estatística Básica** (EP05011) e talvez tenha tido pouco contato com R. Tudo bem. Não tratamos ninguém como programador de partida. Sempre que surgir um comando novo, há uma caixa curta explicando o essencial:

Em R, tudo o que você cria fica guardado num **objeto**. O símbolo `<-` é a seta de atribuição: ela guarda o valor da direita no nome da esquerda.

```
peso_medio <- 245.7 # guarda 245.7 no objeto chamado peso_medio
peso_medio          # digitar o nome mostra o conteúdo
#> [1] 245.7
```

Você lê a caixa uma vez; depois ela vira referência para quando precisar voltar.

O que você vai conseguir fazer ao final

- Planejar uma coleta ou experimento com tamanho amostral justificado.
- Importar e tratar seus próprios dados em um **projeto R** organizado.
- Escolher a análise certa, **checar seus pressupostos** e interpretar a saída.
- Produzir um documento `.qmd` que vira PDF (via Typst) ou página web, pronto para um relatório ou capítulo de TC.

Como navegar

Use o menu lateral para ir aos capítulos. Cada parte corresponde a um pilar. Os **desafios** (última parte) trazem conjuntos de dados reais para você resolver — é onde a IA e o seu julgamento trabalham juntos.

Licença. Material didático de uso aberto para fins educacionais. Cite as fontes da ementa ao reutilizar (ver [Referências](#)).

I

Fundamentos: R, RStudio e o fluxo de trabalho

1	Por que R nesta disciplina	11
1.1	O papel da IA aqui	11
1.2	O que vem a seguir	11
2	RStudio e Projetos	13
2.1	O painel do RStudio em 1 minuto	13
2.2	Projetos: o fim dos arquivos perdidos	13
2.3	Peça à IA, mas confira	13
3	Objetos, dados e pacotes	15
3.1	A seta de atribuição	15
3.2	Vetor: vários valores do mesmo tipo	15
3.3	Data frame: a sua tabela de coleta	15
3.4	Fator: variáveis categóricas	15
3.5	Pacotes: funções extras	16
4	Quarto e saída Typst	17
4.1	O que é Quarto	17
4.2	Anatomia de um arquivo .qmd	17
4.3	Renderizar	17
4.4	Figuras e tabelas com legenda e referência	18
5	IA no RStudio (Posit Assistant)	19
5.1	Como pedir bem	19
5.2	O ciclo: pedir → ler → validar → ajustar	19
5.3	Erros honestos que a IA comete	19
5.4	O que continua sendo só seu	19

1. Por que R nesta disciplina

Você poderia analisar dados numa planilha. Por que aprender R?

Três razões que importam para quem faz pesca e aquicultura:

- **Reprodutibilidade.** Um script R é a receita exata da sua análise. Qualquer pessoa — inclusive você daqui a um ano — consegue rodar de novo e obter o mesmo resultado. Numa planilha, ninguém sabe quais células foram editadas à mão.
- **Escala e repetição.** Coletou dados de 12 estações de manguezal? A mesma análise roda para as 12 com o mesmo código. Chegou a 13ª? Uma linha a mais.
- **Da coleta ao artigo, sem sair do lugar.** Com R + Quarto, o mesmo arquivo que faz a análise gera o relatório, o gráfico do TC e a tabela da dissertação. É o que veremos no Pilar 3.

R é uma **linguagem de programação estatística**. RStudio é o **ambiente** onde você escreve R com conforto (editor, gráficos, ajuda, histórico). Você instala os dois separadamente: primeiro o R, depois o RStudio. O RStudio sozinho não funciona — ele precisa do R por baixo.

1.1 O papel da IA aqui

Antes, aprender R significava decorar funções. Agora, com o **Posit Assistant** dentro do RStudio, o peso muda: você descreve o que quer em português, a IA propõe o código, e o seu trabalho é **entender, validar e ajustar**. Isso não dispensa aprender os conceitos — pelo contrário, exige que você os domine para saber se a sugestão da IA está correta.

Pense na IA como um estagiário muito rápido que digita sem reclamar, mas que **não sabe se os pressupostos da sua análise foram atendidos e não conhece o seu peixe**. Quem decide se a análise faz sentido é você.

1.2 O que vem a seguir

No próximo capítulo, instalamos e conhecemos o RStudio e — o ponto mais importante para organização — os **Projetos do RStudio**, que evitam o caos de arquivos espalhados.

https://www.youtube.com/embed/SEU_VIDEO_ID

2. RStudio e Projetos

2.1 O painel do RStudio em 1 minuto

Ao abrir o RStudio você vê quatro áreas:

- **Editor** (canto superior esquerdo): onde você escreve e salva seus scripts. É aqui que o trabalho fica guardado.
- **Console** (canto inferior esquerdo): onde o R executa e responde. Bom para testar uma linha rápida; ruim para guardar trabalho — o que você digita aqui se perde.
- **Environment** (canto superior direito): lista os objetos que existem agora (seus dados, seus resultados).
- **Output** (canto inferior direito): gráficos, arquivos, pacotes e ajuda.

Regra de ouro: **escreva no Editor, não no Console**. Para enviar uma linha do editor ao console, use **Ctrl+Enter** (Windows/Linux) ou **Cmd+Enter** (Mac). Assim seu trabalho fica salvo no script e ao mesmo tempo executa.

2.2 Projetos: o fim dos arquivos perdidos

O erro nº 1 de quem começa é espalhar dados, scripts e gráficos por pastas diferentes e depender de caminhos como `C:/Usuário/Downloads/dados_finais_v3_REAL.csv`. Quando você muda de computador, nada funciona.

A solução é o **Projeto do RStudio**: uma pasta única que contém tudo de um trabalho. Ao abrir o projeto, o R passa a “morar” ali — todos os caminhos ficam relativos a essa pasta.

Um **Projeto** (arquivo `.Rproj`) define o **diretório de trabalho** automaticamente. Dentro dele você nunca escreve o caminho completo de um arquivo; escreve só `dados/coleta.csv`, e funciona em qualquer máquina. Reprodutibilidade começa aqui.

Estrutura recomendada para os trabalhos desta disciplina:

```
meu-trabalho/  
├─ meu-trabalho.Rproj  
├─ dados/          ← arquivos .csv, .xlsx originais (nunca editados à mão)  
├─ scripts/        ← seus .R  
├─ figuras/        ← gráficos exportados  
└─ relatorio.qmd   ← o documento Quarto (Pilar 3)
```

Para criar: **File** → **New Project** → **New Directory** → **New Project**, escolha o nome e a pasta.

Para ler um arquivo dentro do projeto, use caminho **relativo**:

```
dados <- read.csv("dados/coleta.csv") # ✅ funciona em qualquer computador  
# em vez de  
dados <- read.csv("C:/Users/voce/Downloads/coleta.csv") # ❌ quebra fora da sua  
máquina
```

2.3 Peça à IA, mas confira

Você pode pedir ao Posit Assistant: “crie a estrutura de pastas de um projeto de análise”. Ele sugere comandos. Confira se as pastas fazem sentido para o **seu** trabalho antes de aceitar.

https://www.youtube.com/embed/SEU_VIDEO_ID

3. Objetos, dados e pacotes

Tudo em R é um **objeto**. Entender quatro tipos resolve 90% do dia a dia.

3.1 A seta de atribuição

```
comprimento <- 32.5    # cria um objeto numérico
especie <- "tainha"    # cria um objeto de texto (entre aspas)
```

<- guarda o valor à direita no nome à esquerda. Texto vai sempre entre aspas; número, não. O nome do objeto não pode ter espaço nem começar com número — use `peso_total`, não `peso total`.

3.2 Vetor: vários valores do mesmo tipo

A maioria dos dados biológicos é uma sequência de medidas. Em R, isso é um **vetor**, criado com `c()` (de *combine*):

```
pesos <- c(210, 245, 198, 302, 267)
mean(pesos)      # média
#> [1] 244.4
length(pesos)    # quantos valores
#> [1] 5
```

3.3 Data frame: a sua tabela de coleta

O objeto mais importante é o **data frame**: uma tabela com linhas (observações) e colunas (variáveis), como uma planilha.

Num data frame de pesca, cada **linha** é tipicamente um indivíduo coletado e cada **coluna** é uma variável medida (espécie, comprimento, peso, estação, sexo). Manter esse formato — uma observação por linha, uma variável por coluna — é o que torna a análise simples depois. Esse formato é chamado de **dados arrumados** (*tidy data*).

```
peixes <- data.frame(
  especie = c("tainha", "tainha", "robalo"),
  comp_cm = c(32.5, 28.1, 41.0),
  peso_g  = c(245, 198, 510)
)
peixes
#>   especie comp_cm peso_g
#> 1  tainha   32.5    245
#> 2  tainha   28.1    198
#> 3  robalo   41.0    510
```

Para olhar um data frame: `head(peixes)` mostra as primeiras linhas, `str(peixes)` mostra o tipo de cada coluna, `summary(peixes)` resume.

3.4 Fator: variáveis categóricas

`especie` e `sexo` são **categorias**. Quando vamos analisar grupos, convém transformá-las em **fator**:

```
peixes$especie <- factor(peixes$especie)
levels(peixes$especie)
#> [1] "robalo" "tainha"
```

O `$` acessa uma coluna do data frame pelo nome: `peixes$peso_g` é o vetor de pesos. Você verá esse `$` o tempo todo.

3.5 Pacotes: funções extras

O R básico já faz muita coisa, mas pacotes ampliam suas capacidades. Você **instala uma vez e carrega a cada sessão**:

```
install.packages("tidyverse") # uma vez só, baixa da internet
library(tidyverse)           # toda vez que abrir o R e for usar
```

Os pacotes centrais deste curso:

Pacote	Para quê
<code>tidyverse</code>	importar, limpar e transformar dados (inclui <code>dplyr</code> , <code>ggplot2</code> , <code>readr</code>)
<code>readxl</code>	ler arquivos <code>.xlsx</code> do Excel
<code>car</code>	testes de pressupostos (homogeneidade de variância)
<code>ggplot2</code>	gráficos (vem no tidyverse)
<code>knitr</code> / <code>quarto</code>	gerar relatórios (Pilar 3)

Não sabe qual pacote usar para uma tarefa? Pergunte ao Posit Assistant: “*qual pacote do tidyverse filtra linhas de um data frame?*”. Ele responde `dplyr` e a função `filter()`. Você confirma na documentação (? `filter`) antes de confiar.

https://www.youtube.com/embed/SEU_VIDEO_ID

4. Quarto e saída Typst

Este capítulo é especial: **o guia que você está lendo foi feito em Quarto**. Ao aprender Quarto, você aprende a ferramenta com que este próprio web-book é construído.

4.1 O que é Quarto

Quarto é um sistema que combina, num único arquivo `.qmd`:

- **texto** (escrito em Markdown — fácil, sem formatação complicada),
- **código R** (que roda de verdade e produz resultados),
- e **resultados** (tabelas, gráficos, números) inseridos automaticamente.

Você escreve uma vez e gera o que quiser: página web, PDF, Word. Para esta disciplina, a saída de PDF usa o **Typst**.

Por que Typst e não LaTeX? Typst é um motor de composição moderno, rápido e que não exige a instalação pesada do LaTeX. Para relatórios de aula, TCs e artigos, ele produz um PDF limpo com muito menos dor de cabeça. O Quarto cuida de converter seu `.qmd` em Typst automaticamente.

4.2 Anatomia de um arquivo `.qmd`

Todo `.qmd` tem um **cabeçalho YAML** no topo, entre `---`, que define título e formato de saída:

```
---
title: "Relatório de aula prática – Biometria de tainhas"
author: "Seu Nome"
format: typst
---

## Introdução

Coletamos 30 indivíduos no estuário do rio Caeté.

::: {.cell}

```{r .cell-code}
peixes <- read.csv("dados/coleta.csv")
mean(peixes$peso_g)
```

:::

O peso médio observado aparece automaticamente acima.
```

O bloco entre ````{r}` e ````` é um **chunk** de código. Quando você renderiza o documento, o R executa o chunk e insere o resultado no PDF. Você nunca copia e cola números à mão — eles vêm direto da análise. Se os dados mudarem, o relatório se atualiza sozinho.

4.3 Renderizar

No RStudio, abra o `.qmd` e clique no botão **Render** (ou `Ctrl+Shift+K`). Com `format: typst`, sai um PDF. Com `format: html`, sai uma página web.

```
# também funciona pelo console:  
quarto::quarto_render("relatorio.qmd")
```

4.4 Figuras e tabelas com legenda e referência

```
::: {.cell}  
  
```.r .cell-code  
hist(peixes$peso_g)
```.br/>  
:::
```

Como mostra a @fig-pesos, os pesos concentram-se entre 200 e 300 g.

O @fig-pesos cria uma **referência cruzada** numerada automaticamente — exatamente o que um TC ou artigo precisa.

Peça à IA: “escreva o cabeçalho YAML de um relatório Quarto com saída Typst, título X e meu nome”. Ela entrega o esqueleto. Você preenche o conteúdo científico — que é a parte que importa e que só você pode escrever.

https://www.youtube.com/embed/SEU_VIDEO_ID

5. IA no RStudio (Posit Assistant)

A partir de 2026 o RStudio traz IA integrada. Este capítulo é o **manual de uso consciente** dela nesta disciplina.

5.1 Como pedir bem

Uma IA responde tão bem quanto a sua pergunta. Compare:

✗ “faz um gráfico”

✓ “faça um boxplot do peso (`peso_g`) por espécie (`especie`) usando ggplot2, com eixos rotulados em português”

A segunda menciona **as variáveis reais, o pacote e o que quer ver**. O resultado é muito melhor. Um bom pedido à IA tem três partes: **o que** (boxplot), **com quais dados** (colunas `peso_g` e `especie`), **com qual ferramenta** (ggplot2). Quanto mais específico, menos você precisa corrigir depois.

5.2 O ciclo: pedir → ler → validar → ajustar

1. **Pedir** o código.
2. **Ler** linha por linha. Se houver um comando que você não entende, peça à IA para explicá-lo — ou consulte a caixa “R na prática” correspondente neste guia.
3. **Validar**: o resultado faz sentido biológico? Os pressupostos da análise foram checados? (ver o capítulo de [Pressupostos](#)).
4. **Ajustar**: peça correções específicas.

A IA vai, com toda confiança, rodar um teste-t entre dois grupos sem nunca perguntar se os dados são normais ou se as variâncias são iguais. **Esse silêncio é o seu trabalho**. Antes de aceitar qualquer teste sugerido, pergunte-se: *quais são os pressupostos deste teste e eu os verifiquei?* É exatamente o que avaliamos nesta disciplina — não a sua capacidade de digitar o código, que a IA já faz.

5.3 Erros honestos que a IA comete

- Usa um teste cujos pressupostos foram violados (o mais perigoso, porque o código *roda* e dá um número).
- Inventa nomes de função ou de coluna que não existem nos seus dados.
- Mistura dialetos (R base com tidyverse) de um jeito que confunde.
- Não conhece o seu peixe: pode sugerir uma análise estatisticamente válida, mas sem sentido para a biologia da espécie.

5.4 O que continua sendo só seu

Planejar a coleta, escolher a pergunta científica, julgar os pressupostos, interpretar o resultado à luz da biologia e da ecologia, e decidir o que comunicar. A IA acelera a digitação; o pensamento estatístico e biológico é seu.

https://www.youtube.com/embed/SEU_VIDEO_ID

II

Pilar 1 — Planejamento amostral e experimental

6	Pilar 1 — Por que planejar antes de coletar	23
6.1	As perguntas do planejamento	23
6.2	O que vem nos próximos capítulos	23
7	Amostragem em campo e entrevistas	25
7.1	Tipos de amostragem	25
7.2	Sortear no R	25
7.3	Entrevistas com pescadores e comunidades . . .	25
7.4	Tamanho amostral: a ideia	25
8	Planejamento de experimentos	27
8.1	Pseudo-replicação: o erro clássico	27
8.2	Delineamentos comuns	27
8.3	Casualizar no R	27
8.4	Do delineamento à análise	27

6. Pilar 1 — Por que planejar antes de coletar

A estatística não começa na análise. Começa **antes de ir a campo**. A decisão mais cara de um trabalho é coletar dados que não respondem à pergunta — e isso só se evita planejando.

População é o conjunto sobre o qual você quer concluir (por exemplo, todas as tainhas do estuário do Caeté). **Amostra** é a parte que você efetivamente mede. A estatística é a ponte que permite, a partir da amostra, dizer algo sobre a população — mas só se a amostra for obtida de forma adequada. Uma amostra mal planejada produz conclusões erradas por mais sofisticada que seja a análise depois.

6.1 As perguntas do planejamento

Antes de qualquer coleta, responda:

1. **Qual é a pergunta?** “O peso difere entre dois estuários?” é diferente de “qual a relação peso-comprimento?”. A pergunta define a análise, e a análise define o que coletar.
2. **Qual a unidade amostral?** Um indivíduo? Uma estação? Uma embarcação? Um pescador entrevistado?
3. **Quantas unidades?** É o **tamanho amostral** — nem tão pequeno que não detecte o efeito, nem tão grande que desperdice recurso.
4. **Como sortear?** Aleatório? Estratificado por estação? Sistemático ao longo de um transecto?

6.2 O que vem nos próximos capítulos

- **Amostragem em campo e entrevistas:** tipos de amostragem, como sortear, e cuidados ao entrevistar pescadores.
- **Planejamento de experimentos:** controle, repetição, casualização — para experimentos com pescado e ecológicos.

A IA pode calcular um tamanho amostral se você lhe der o efeito esperado, a variabilidade e o poder desejado. O que ela **não** faz é definir a sua pergunta científica nem dizer qual é a unidade amostral correta para o seu peixe — isso depende de você conhecer o sistema.

7. Amostragem em campo e entrevistas

7.1 Tipos de amostragem

- **Aleatória simples:** cada unidade tem a mesma chance de ser sorteada. Simples, mas pode deixar áreas inteiras de fora por acaso.
- **Estratificada:** divide a população em grupos homogêneos (*estratos* — por exemplo, estações de manguezal, classes de profundidade) e sorteia dentro de cada um. Garante representação de todos os grupos e costuma ser a melhor escolha em ambientes heterogêneos como estuários.
- **Sistemática:** sorteia o primeiro e depois pega a cada k unidades (a cada 10 m de um transecto, a cada 5ª embarcação). Prática em campo, mas cuidado se houver um padrão cíclico no ambiente.
- **Por conglomerados:** sorteia grupos inteiros (todas as embarcações de um porto). Útil quando listar todas as unidades é inviável.

7.2 Sortear no R

Sortear de verdade — em vez de “escolher a olho” — elimina o viés do coletor.

```
# sortear 8 estações de uma lista de 20, sem repetição
estacoes <- paste0("E", 1:20)
set.seed(42) # garante que o sorteio seja reproduzível
sample(estacoes, size = 8)
#> [1] "E1" "E5" "E10" "E12" "E14" "E15" "E18" "E19"
```

`set.seed()` fixa o ponto de partida do sorteio aleatório. Com a mesma semente, o `sample()` devolve sempre o mesmo resultado — essencial para que outra pessoa reproduza o seu sorteio. Use sempre antes de qualquer comando aleatório que precise ser documentado.

7.3 Entrevistas com pescadores e comunidades

Quando a “amostra” são pessoas, o planejamento ganha camadas:

- **Quem é a população?** Pescadores cadastrados na colônia? Todos que desembarcam no porto? A definição muda quem você pode entrevistar.
- **Questionário estruturado** facilita a análise depois: perguntas fechadas viram colunas categóricas; evite perguntas ambíguas.
- **Ética:** consentimento informado e anonimização. Dados de pessoas exigem cuidado adicional.
- **Viés de não-resposta:** quem se recusa a responder pode diferir de quem responde. Registre as recusas.

7.4 Tamanho amostral: a ideia

O tamanho amostral adequado depende de três coisas: **quão grande é o efeito** que você quer detectar, **quanta variabilidade** existe nos dados, e **qual risco de erro** você aceita (poder estatístico). Mais variabilidade ou efeitos menores exigem amostras maiores. Não existe um número mágico universal — “30” é uma lenda.

Cálculo de poder para comparar duas médias:

```
# install.packages("pwr")
library(pwr)
pwr.t.test(d = 0.8, sig.level = 0.05, power = 0.80)
# d = tamanho de efeito esperado (diferença padronizada)
# devolve o n necessário por grupo
```

Peça à IA: “*quantos indivíduos por grupo preciso para detectar uma diferença de 0,8 desvios-padrão com poder de 80%?*”. Ela usa `pwr`. Mas só você sabe se 0,8 é uma diferença biologicamente relevante para a sua espécie — esse valor não sai de uma fórmula, sai do conhecimento do sistema.

8. Planejamento de experimentos

Em experimentos com pescado (crescimento sob dietas, densidades de estocagem) ou ecológicos (efeito de um tratamento sobre uma comunidade), três princípios protegem suas conclusões.

Os três princípios de Fisher:

- **Repetição** — cada tratamento aplicado a várias unidades experimentais independentes. Sem repetição, você não consegue estimar a variabilidade natural e não sabe se a diferença observada é real ou acaso.
- **Casualização** — sortear quais unidades recebem qual tratamento. Evita que um viés (um tanque mais ensolarado, por exemplo) se confunda com o efeito do tratamento.
- **Controle local (blocagem)** — agrupar unidades parecidas em blocos e aplicar todos os tratamentos dentro de cada bloco. Remove a variação conhecida (prateleira, dia, lote) para enxergar melhor o efeito do tratamento.

8.1 Pseudo-replicação: o erro clássico

Se você tem **um** tanque por tratamento e mede 20 peixes dentro dele, você **não** tem 20 repetições — tem 1. Os 20 peixes compartilham o mesmo tanque, então não são independentes. Isso se chama **pseudo-replicação** e é um dos erros mais comuns (e mais graves) em experimentos de aquicultura e ecologia. A unidade experimental é o tanque, não o peixe.

8.2 Delineamentos comuns

- **Inteiramente casualizado (DIC)**: tratamentos sorteados livremente entre as unidades. Bom quando as unidades são homogêneas (tanques idênticos, mesma sala).
- **Blocos casualizados (DBC)**: quando há uma fonte de variação conhecida (prateleiras, dias de coleta), bloca-se por ela.
- **Fatorial**: testa dois fatores ao mesmo tempo (dieta × temperatura) e, de quebra, a **interação** entre eles.

8.3 Casualizar no R

```
# sortear 12 tanques entre 3 dietas (4 repetições cada) — DIC
tanques <- 1:12
dietas <- rep(c("A", "B", "C"), each = 4)
set.seed(7)
data.frame(tanque = tanques, dieta = sample(dietas))
```

8.4 Do delineamento à análise

O delineamento **decide a análise**. Não é o contrário. Anote:

Delineamento	Análise típica
DIC, 1 fator, 2 grupos	teste-t
DIC, 1 fator, 3+ grupos	ANOVA de uma via
DBC	ANOVA com bloco
Fatorial 2 fatores	ANOVA fatorial (com interação)

Veremos cada uma no [capítulo de análises](#) — sempre depois de checar os [pressupostos](#).

A IA roda a ANOVA do seu fatorial sem piscar. O que ela não faz por você: garantir que você teve repetição verdadeira (não pseudo-replicação) e que casualizou. Esses cuidados acontecem **antes** do código, no campo e no planejamento — e nenhuma análise conserta sua ausência.

III

Pilar 2 — Análise com R

9	Pilar 2 — Importar dados	31
9.1	Antes de importar: organize a planilha	31
9.2	Importar CSV	31
9.3	Importar Excel (.xlsx)	31
9.4	A primeira coisa a fazer depois de importar . . .	31
10	Tratamento de dados	33
10.1	O operador pipe	33
10.2	Os verbos do dplyr	33
10.3	Resumir por grupo	33
10.4	Lidar com valores ausentes e categorias inconsistentes	33
10.5	O resultado tratado alimenta o Pilar 3	34
11	Pressupostos das análises	35
11.1	Os pressupostos mais comuns	35
11.2	Verificar normalidade	35
11.3	Verificar homogeneidade de variâncias	35
11.4	E quando os pressupostos falham?	35
11.5	O fluxo correto	36
12	As análises	37
12.1	Mapa de decisão	37
12.2	Comparar dois grupos	37
12.3	Comparar três ou mais grupos: ANOVA	37
12.4	Relação entre variáveis numéricas: regressão . .	37
12.5	Variáveis categóricas: qui-quadrado	38
12.6	Reportar o resultado	38

9. Pilar 2 — Importar dados

Com o planejamento feito e os dados coletados, começa a análise. E ela começa por trazer os dados para dentro do R — sem digitar nada à mão.

9.1 Antes de importar: organize a planilha

A maior parte dos problemas de importação nasce de planilhas bagunçadas. Antes de salvar:

Planilha pronta para análise (dados arrumados):

- Uma **observação por linha**, uma **variável por coluna**.
- **Primeira linha** = nomes das colunas, curtos, sem espaço nem acento (`comp_cm`, não `Comprimento (cm)`).
- **Sem células mescladas**, sem títulos no meio, sem cores que “significam” algo.
- Valores ausentes ficam **em branco** ou marcados como `NA` — nunca com “?”, “-” ou “sem dado”.
- Decimais com **ponto** (`32.5`), não vírgula, para evitar dor de cabeça.

9.2 Importar CSV

```
library(readr)
peixes <- read_csv("dados/coleta.csv")
```

9.3 Importar Excel (.xlsx)

```
library(readxl)
peixes <- read_excel("dados/coleta.xlsx", sheet = "biometria")
```

Repare no caminho `"dados/coleta.csv"`: é **relativo ao projeto** (Capítulo 2). Como você está dentro do Projeto do RStudio, não precisa do caminho completo. Isso faz o código funcionar em qualquer computador.

9.4 A primeira coisa a fazer depois de importar

Sempre inspecione antes de analisar:

```
head(peixes)      # primeiras linhas — bateu com a planilha?
str(peixes)       # tipo de cada coluna — número é número? texto é texto?
summary(peixes)   # resumo — há valores absurdos? NAs demais?
```

Se uma coluna que deveria ser número aparece como texto (`chr`), quase sempre é vírgula decimal ou um caractere estranho escondido. Resolva antes de seguir.

“Importei meus dados mas a coluna peso virou texto, por quê?” — a IA costuma diagnosticar (vírgula decimal, espaço, símbolo). Ótimo uso. Mas confira você mesmo a planilha original; às vezes o problema é um dado digitado errado lá na fonte, e nenhum código conserta isso melhor que corrigir a origem.

10. Tratamento de dados

Dados reais chegam sujos: valores faltando, categorias escritas de três jeitos, colunas a calcular. Tratar é prepará-los para a análise — e para o Pilar 3.

10.1 O operador pipe

O **pipe** `|>` encadeia operações: pega o resultado da esquerda e passa como entrada para a direita. Leia como “e então”.

```
peixes |>
  filter(especie == "tainha") |> # e então fique só com tainhas
  summarise(peso_medio = mean(peso_g)) # e então calcule a média
```

É muito mais legível que aninhar funções uma dentro da outra.

10.2 Os verbos do dplyr

Cinco funções resolvem quase tudo:

Verbo	Faz	Exemplo
<code>filter()</code>	mantém linhas que satisfazem uma condição	<code>filter(peso_g > 200)</code>
<code>select()</code>	escolhe colunas	<code>select(especie, peso_g)</code>
<code>mutate()</code>	cria/altera colunas	<code>mutate(peso_kg = peso_g / 1000)</code>
<code>arrange()</code>	ordena linhas	<code>arrange(desc(peso_g))</code>
<code>summarise()</code>	resume (com <code>group_by</code>)	<code>summarise(media = mean(peso_g))</code>

10.3 Resumir por grupo

A combinação mais útil — estatística descritiva por espécie ou estação:

```
peixes |>
  group_by(especie) |>
  summarise(
    n = n(),
    peso_medio = mean(peso_g, na.rm = TRUE),
    desvio = sd(peso_g, na.rm = TRUE)
  )
```

`na.rm = TRUE` diz ao R para **ignorar valores ausentes (NA)** no cálculo. Sem isso, se houver um único NA na coluna, a média devolve NA. Quase toda função de resumo aceita esse argumento.

10.4 Lidar com valores ausentes e categorias inconsistentes

```
# quantos NA por coluna?
colSums(is.na(peixes))
```

```
# padronizar categorias escritas de formas diferentes
peixes <- peixes |>
  mutate(especie = tolower(trimws(especie))) # minúsculas + sem espaços nas pontas
```

10.5 O resultado tratado alimenta o Pilar 3

Guarde o data frame limpo. É ele que entra nos gráficos, nas tabelas e no relatório Quarto. Um princípio: **nunca edite o arquivo original à mão** — todo tratamento é feito por código, para que seja reproduzível e auditável.

Tratamento de dados é onde a IA mais ajuda: “padronize a coluna especie para minúsculas e remova espaços”, “crie uma coluna de classe de tamanho a cada 5 cm”. Ela escreve o `mutate()` correto. Você confere o resultado com `head()` e `summary()` — porque uma regra de limpeza errada estraga silenciosamente todos os dados seguintes.

11. Pressupostos das análises

Este é o capítulo mais importante do curso. É **aqui que mora a competência que a IA não tem**. Um teste estatístico só é válido se seus pressupostos forem atendidos. Rodar o teste é fácil; saber se ele *podia* ser rodado é o que separa uma análise correta de um número sem sentido.

A IA roda qualquer teste que você pedir e devolve um p-valor com toda a confiança do mundo — **mesmo quando os pressupostos foram violados e o p-valor não significa nada**. Verificar pressupostos é, por isso, a sua responsabilidade central nesta disciplina. Não delegue.

11.1 Os pressupostos mais comuns

A maioria dos testes “clássicos” (teste-t, ANOVA, regressão linear) assume:

- **Independência das observações** — uma medida não influencia a outra. Vem do **planejamento**, não dá para testar depois. Pseudo-replicação (Cap. 8) viola isso.
- **Normalidade** — os dados (mais precisamente, os resíduos) seguem aproximadamente uma distribuição normal.
- **Homogeneidade de variâncias (homocedasticidade)** — os grupos comparados têm variabilidade parecida.

A independência é a mais importante e a única que não se conserta com matemática — só com bom planejamento.

11.2 Verificar normalidade

Combine **gráfico** (mais informativo) e **teste**:

```
# gráfico Q-Q: pontos próximos da linha = aproximadamente normal
qqnorm(peixes$peso_g); qqline(peixes$peso_g)

# histograma
hist(peixes$peso_g)

# teste de Shapiro-Wilk: H0 = dados são normais
shapiro.test(peixes$peso_g)
```

Cuidado com a interpretação do teste de normalidade: com amostra **grande**, ele detecta desvios mínimos e “rejeita” a normalidade mesmo quando ela é boa o bastante na prática; com amostra **pequena**, ele quase nunca rejeita. Por isso o **gráfico Q-Q** costuma decidir melhor que o p-valor. Olhe a forma, não só o número.

11.3 Verificar homogeneidade de variâncias

```
# install.packages("car")
library(car)
leveneTest(peso_g ~ especie, data = peixes) # H0 = variâncias iguais
```

11.4 E quando os pressupostos falham?

Você tem caminhos — e escolher entre eles é decisão sua:

- **Transformar** os dados (log, raiz quadrada) e reverificar.
- **Usar um teste que não exige normalidade** (não-paramétrico): Mann-Whitney no lugar do teste-t, Kruskal-Wallis no lugar da ANOVA.

- **Usar modelos que acomodam a estrutura** (modelos lineares generalizados), em situações mais avançadas.

```
# alternativa não-paramétrica ao teste-t  
wilcox.test(peso_g ~ especie, data = peixes)
```

11.5 O fluxo correto

1. Defina a pergunta e o teste candidato (vem do delineamento, Cap. 8).
2. **Cheque os pressupostos** desse teste.
3. Se atendidos → rode o teste. Se não → transforme ou troque por uma alternativa, e cheque de novo.
4. Só então interprete o resultado.

Use a IA assim: “*verifique os pressupostos de uma ANOVA de peso_g por especie nestes dados*” — ela gera o Q-Q plot, o Levene e o Shapiro. Perfeito. Mas a frase “*os pressupostos foram atendidos?*” você responde **você**, lendo os gráficos. Se a IA disser “tudo certo, pode rodar”, desconfie e confira: essa é a parte do trabalho que é sua.

12. As análises

Com os pressupostos checados, escolhemos a análise. A pergunta e o delineamento decidem — não o contrário.

12.1 Mapa de decisão

Que análise usar?

- Comparar a **média de 2 grupos** → teste-t (ou Mann-Whitney se a normalidade falhar).
- Comparar a **média de 3+ grupos** → ANOVA (ou Kruskal-Wallis).
- **Relação entre duas variáveis numéricas** → correlação e regressão linear.
- **Associação entre duas variáveis categóricas** → qui-quadrado.
- **Dois fatores ao mesmo tempo** → ANOVA fatorial (testa também a interação).

12.2 Comparar dois grupos

```
# pressupostos OK → teste-t
t.test(peso_g ~ especie, data = peixes)

# normalidade falhou → alternativa não-paramétrica
wilcox.test(peso_g ~ especie, data = peixes)
```

12.3 Comparar três ou mais grupos: ANOVA

```
modelo <- aov(peso_g ~ estuario, data = peixes)
summary(modelo)

# se a ANOVA for significativa, qual grupo difere de qual?
TukeyHSD(modelo)
```

A fórmula `peso_g ~ estuario` lê-se “**peso_g em função de estuario**”. O `~` aparece em quase todas as análises do R: à esquerda a variável resposta, à direita a(s) explicativa(s). É a gramática central da modelagem em R.

12.4 Relação entre variáveis numéricas: regressão

A relação peso–comprimento é o pão com manteiga da biologia pesqueira:

```
modelo <- lm(peso_g ~ comp_cm, data = peixes)
summary(modelo)      # coeficientes, R², p-valor
```

A regressão **também tem pressupostos**, verificados nos **resíduos** (não nos dados brutos): os resíduos devem ser aproximadamente normais, ter variância constante e não mostrar padrão. Cheque com `plot(modelo)` — quatro gráficos diagnósticos aparecem. Para peso–comprimento, costuma-se ajustar a relação na forma log–log, mas isso fica para a aula.

```
plot(modelo)  # diagnóstico dos resíduos — sempre olhe antes de confiar no R²
```

12.5 Variáveis categóricas: qui-quadrado

```
tabela <- table(peixes$sexo, peixes$maturacao)
chisq.test(tabela)
```

12.6 Reportar o resultado

Um resultado completo traz: o **teste usado**, a **estatística**, os **graus de liberdade**, o **p-valor** e — fundamental — o **tamanho do efeito** ou a estimativa em unidades reais. Um p-valor sozinho diz se há efeito, não o quão grande ele é. Para a biologia, o tamanho importa tanto quanto a significância.

A IA escreve `t.test(...)` e `aov(...)` perfeitamente e até redige uma frase de resultado. O seu trabalho: confirmar que era esse o teste certo (pressupostos!), que a fórmula reflete o seu delineamento, e que a interpretação faz sentido biológico. A IA não sabe que um peso médio de 2 kg é absurdo para a sua espécie de 20 cm — você sabe.

IV

Pilar 3 — Comunicação de resultados

13	Pilar 3 — Comunicar resultados	41
13.1	O princípio do documento único	41
13.2	Estrutura de um relatório de aula prática	41
13.3	Tabelas e figuras que se referenciam sozinhas . .	42
13.4	Citar a literatura	42
13.5	Renderizar para entrega	42

13. Pilar 3 — Comunicar resultados

A análise só tem valor quando comunicada. Este pilar transforma o seu data frame analisado naquilo que vai no **relatório de aula, no TC, na dissertação, na tese ou no artigo** — usando Quarto com saída Typst, o mesmo sistema que constrói este guia.

13.1 O princípio do documento único

Num documento Quarto, **a análise e o texto vivem no mesmo arquivo**. Você não copia números da análise para o relatório: o relatório executa a análise e insere os resultados sozinho. Se um dado mudar, basta renderizar de novo — números, tabelas e gráficos se atualizam. Isso elimina a classe inteira de erros de “copiei o valor errado da planilha”.

13.2 Estrutura de um relatório de aula prática

Reproveite o esqueleto a cada trabalho:

```
---
title: "Biometria de tainhas no estuário do Caeté"
author: "Seu Nome"
date: today
format: typst
bibliography: referencias.bib
csl: https://www.zotero.org/styles/abnt
---

# Introdução
Contextualização e objetivo.

# Material e Métodos
Onde, quando, como coletou; qual análise e por quê.

# Resultados

::: {#tbl-descritiva .cell tbl-cap='Estatística descritiva por espécie.'}

```.r .cell-code
peixes |>
 group_by(especie) |>
 summarise(n = n(), media = mean(peso_g), dp = sd(peso_g))
```

:::

::: {.cell}

```.r .cell-code
ggplot(peixes, aes(especie, peso_g)) + geom_boxplot()
```

:::
```

```
A @tbl-descritiva resume os dados; a @fig-boxplot mostra que...

# Discussão
Interprete à luz da biologia e da literatura [@sokal2012].

# Referências
```

13.3 Tabelas e figuras que se referenciam sozinhas

- `#| label: tbl-... + #| tbl-cap:` → tabela numerada; cite com `@tbl-...`
- `#| label: fig-... + #| fig-cap:` → figura numerada; cite com `@fig-...`
- `#| include: false` → roda o código mas não mostra (bom para o `setup`).
- `#| echo: false` → mostra o resultado mas esconde o código (bom para relatório final).

As opções que começam com `#|` dentro de um chunk são **opções do chunk**. Elas controlam o que aparece no documento final. Num relatório científico você quase sempre quer `#| echo: false` nos chunks de resultado — o leitor vê o gráfico, não o código que o gerou.

13.4 Citar a literatura

As referências ficam num arquivo `referencias.bib` (você exporta do Zotero ou Mendeley). No texto, `[@sokal2012]` vira a citação formatada; o estilo ABNT (definido no `cs1`) cuida da formatação exigida pelo TC e pela dissertação. A lista de referências é montada automaticamente no fim.

13.5 Renderizar para entrega

- **PDF** para entregar (relatório, TC): `format: typst` → botão **Render**.
- **HTML** para publicar na web (como este guia): `format: html`.

Peça à IA: “*transforme esta análise num relatório Quarto com seções de Métodos e Resultados, código escondido e tabela com legenda*”. Ela monta o esqueleto e as opções de chunk. O conteúdo científico — a interpretação, a discussão, a conexão com a literatura — é seu, e é o que faz o trabalho valer.



Desafios com dados

| | | |
|-------------|--|-----------|
| 14 | Desafios com dados | 45 |
| 14.1 | Como entregar um desafio | 45 |
| 14.2 | Desafio 1 — Peso entre dois estuários (nível inicial) | 45 |
| 14.3 | Desafio 2 — Dieta e crescimento (nível intermediário) | 45 |
| 14.4 | Desafio 3 — Relação peso-comprimento (nível intermediário) | 45 |
| 14.5 | Desafio 4 — Percepção dos pescadores (dados de entrevista) | 45 |
| 14.6 | Autoavaliação | 45 |
| 14.6 | Referências | 47 |
| 14.6 | Livros-guia da ementa | 47 |
| 14.6 | Bibliografia | 49 |

14. Desafios com dados

Aqui você pratica de verdade. Cada desafio traz um conjunto de dados (na pasta `dados/` do projeto) e uma pergunta. Você resolve do começo ao fim: importar, tratar, **checar pressupostos**, analisar e comunicar — usando a IA como apoio e o seu julgamento como guia.

Regra dos desafios: você pode usar o Posit Assistant à vontade para escrever código. Mas a entrega precisa mostrar que **você** decidiu o teste, **você** verificou os pressupostos e **você** interpretou. Na defesa do desafio, espere a pergunta: “*por que este teste e não outro?*”. A resposta não pode ser “a IA sugeriu”.

14.1 Como entregar um desafio

Um único arquivo `.qmd` que renderiza para PDF (Typst) contendo:

1. A **pergunta** reescrita com suas palavras.
2. O **código** de importação e tratamento.
3. A **verificação de pressupostos** (com gráficos) e a sua decisão sobre o teste.
4. A **análise** e o **resultado**.
5. Uma **interpretação** curta: o que isso significa para a biologia/pesca.

14.2 Desafio 1 — Peso entre dois estuários (nível inicial)

O peso médio das tainhas difere entre o estuário A e o estuário B?

Caminho sugerido: descritiva por grupo → checar normalidade e variâncias → teste-t **ou** Mann-Whitney → interpretar. Dados: `dados/desafio01_estuarios.csv`.

14.3 Desafio 2 — Dieta e crescimento (nível intermediário)

Três dietas produzem ganhos de peso diferentes? Atenção: cada dieta foi aplicada a vários tanques.

Cuidado com a **unidade experimental** (Cap. 8): a repetição é o tanque, não o peixe. Caminho: definir a unidade correta → ANOVA → Tukey → interpretar. Dados: `dados/desafio02_dietas.csv`.

14.4 Desafio 3 — Relação peso–comprimento (nível intermediário)

Qual a relação entre peso e comprimento na espécie X?

Caminho: gráfico de dispersão → regressão → **diagnóstico dos resíduos** → interpretar os coeficientes biologicamente. Dados: `dados/desafio03_biometria.csv`.

14.5 Desafio 4 — Percepção dos pescadores (dados de entrevista)

A percepção sobre a redução do estoque está associada ao tempo de profissão?

Caminho: tratar dados categóricos → tabela de contingência → qui-quadrado → interpretar com cautela ética. Dados: `dados/desafio04_entrevistas.csv`.

14.6 Autoavaliação

Ao terminar cada desafio, pergunte-se:

- Eu saberia explicar **por que** escolhi este teste?
- Eu **verifiquei** os pressupostos antes de rodar?

- Se a IA tivesse sugerido o teste errado, eu teria percebido?

Se respondeu “sim” às três, o objetivo da disciplina foi atingido.

Referências

Livros-guia da ementa

Esta seção lista a bibliografia da disciplina. **Substitua pelos títulos exatos da sua ementa (EP05042)** — os itens abaixo são exemplos comuns da área e estão pré-cadastrados em [referencias.bib](#) para você ajustar.

- Estatística com aplicação biológica e ecológica.
- Delineamento experimental e análise de experimentos.
- Análise de dados ecológicos em R.
- Biometria pesqueira e dinâmica de populações.

Para citar no texto dos capítulos, use `[@chave]` com a chave definida no arquivo [referencias.bib](#). O estilo ABNT é aplicado automaticamente.

Bibliografia